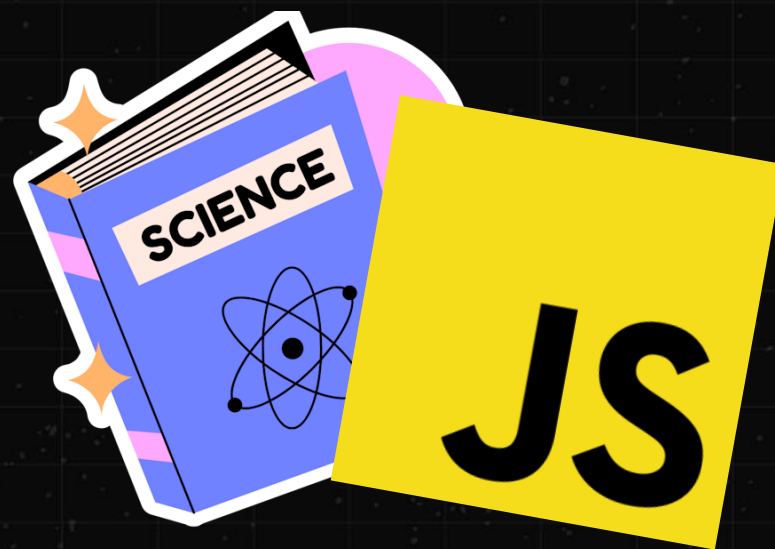


ЛЕКЦИЯ #5

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Будем говорить про **связку процессора и памяти**
- * Как процессор **оптимизируют работу с памятью** с помощью кэша
- * Что такое **предсказание ветвлений**
и как это связано с производительностью нашего кода

КАКОЙ КОД ОТРАБОТАЕТ БЫСТРЕЕ?

```
function rowMajor(array, size) {  
  let sum = 0;  
  
  for (let i = 0; i < size; i++) {  
    for (let j = 0; j < size; j++) {  
      const index = i * size + j;  
      sum += array[index];  
    }  
  }  
  
  return sum;  
}
```

```
function columnMajor(array, size) {  
  let sum = 0;  
  
  for (let i = 0; i < size; i++) {  
    for (let j = 0; j < size; j++) {  
      const index = j * size + i;  
      sum += array[index];  
    }  
  }  
  
  return sum;  
}
```

УДИВИТЕЛЬНО, НО ФАКТ

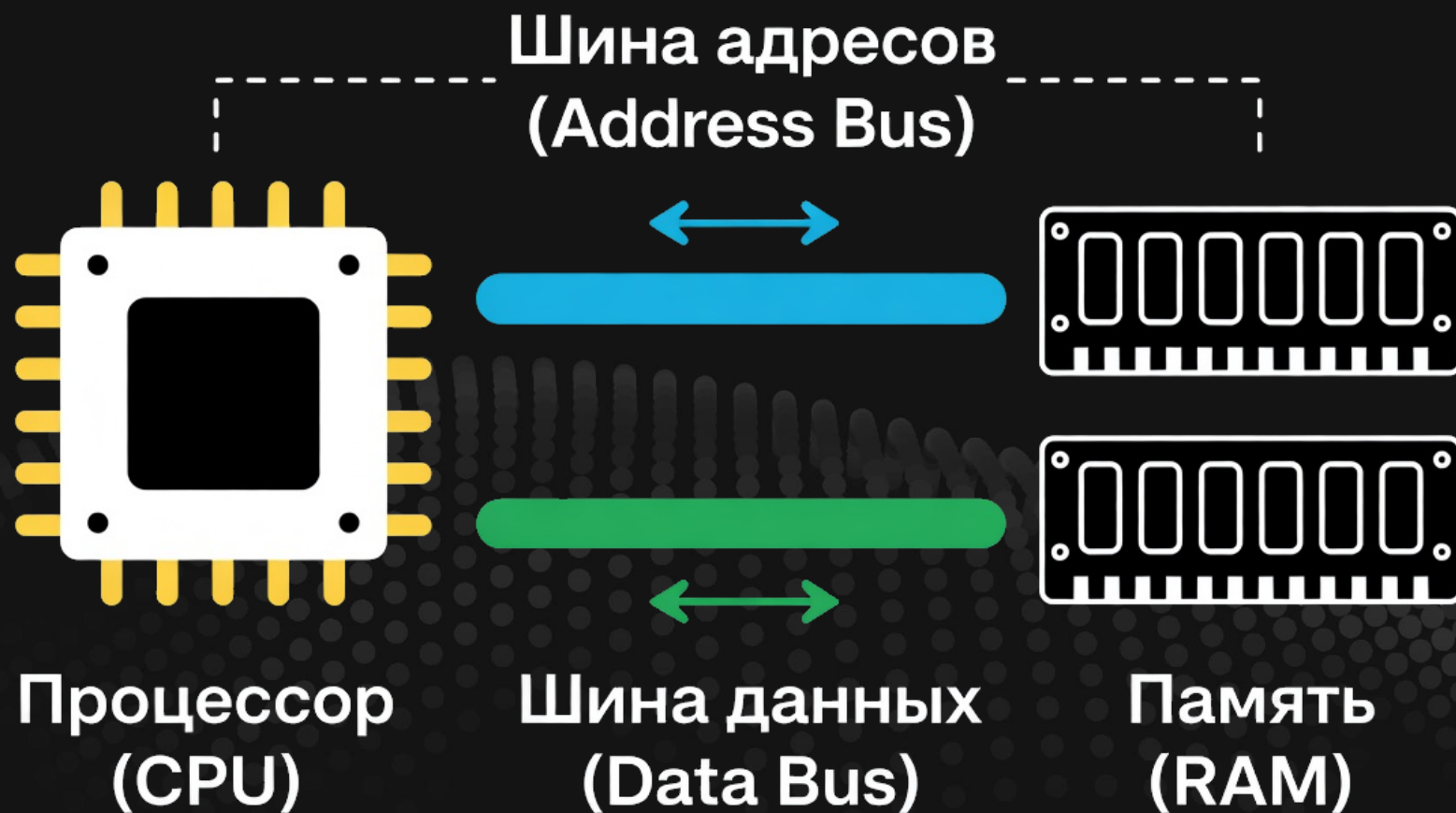
- * Функция `rowMajor` стабильно быстрее в 1.5-2 раза на числовом массиве
- * А с массивом объектов уже быстрее в 3-5 раза!
- * И там и там делается полный обход массива – **разница лишь в порядке обхода**
- * Чем больше массив – тем разница больше

ОТСЕЧЕМ НЕПРАВИЛЬНЫЕ ВЫВОДЫ

- * Доступ к элементу массива по индексу – константа
- * Значит `array[0]` или `array[100]` работают с одинаковой эффективностью
- * Оперативная память в абсолютном большинстве современных компьютеров является DRAM (Dynamic Random Access Memory)
- * Чтение разных ячеек памяти по их адресу также константная операция
- * В чем же тогда дело?

ПРОЦЕССОР И ПАМЯТЬ

- * Это два разных электронных устройства, которые соединяются друг с другом по средствам специальных шин на материнской плате
- * Процессор может считать или записать любой байт оперативной памяти по её адресу
- * Процессор работает со своей скоростью, а память со своей
- * Передача данных тоже не бесплатна и занимает время



🔍 ЧТЕНИЕ



🔥 ЗАПИСЬ



ЕЩЕ ВАЖНЫЙ НЮАНС

- * С точки зрения процессора, вся память – это **большой массив ячеек размером в один байт**
- * Процессор может **обратиться к любой ячейке** по её адресу (индексу в массиве) используя адресную шину и получить/передать байты по шине данных
- * Адрес представляет собой **обычное беззнаковое число в прямом коде**
- * А размер адресной шины **определяет максимально возможное число**
- * Поэтому 32-битная адресная шина **не может адресовать больше 4 Гибибайт памяти**

ЧТО ЗНАЧИТ
НЕ СМОЖЕТ
АДРЕСОВАТЬ?



ЯЧЕЙКИ ПАМЯТИ С АДРЕСОМ БОЛЬШЕ ЭТОГО ЧИСЛА ПРОСТО НЕДОСТУПНЫ

- * В этом кроется причина, почему на процессорах с 16-битной адресной шиной нет возможности подключить больше 64 Киббайт памяти
- * Например, такая адресная шина была у [Intel 8080](#)
- * С 20 битами не более 1 Мебибайт ([Intel 8086](#))
- * На 32 битах не более 4 Гиббайт
- * В современных 64 битных процессорах оно стало немного сложнее

ЧТО **ЗНАЧИТ**
СТАЛО СЛОЖНЕЕ?



ТОГДА И СЕЙЧАС

- * Старые компьютеры использовали **отдельные физические шины адресов и данных**
- * И **специальный контроллер на материнской плате** (северный мост) для синхронизации работы процессора и памяти
- * В современных компьютерах **контроллер переключался на сам процессор**
- * А адреса и данные **физически используют одну шину** (мультиплексирование)
- * Управляет этим процессом контроллер памяти

ФОРМАЛЬНО И ФАКТИЧЕСКИ

- * Размер шины большинства современных процессоров – 64 бита
- * **Теоретически** это позволит адресовать 16 эксбибайт (2 в 64-й степени байт)
- * Практически это значение **значительно ниже**
- * Например, у Ryzen 9950X3D это всего 256 гигабайт (2 в 38 степени)

РАЗМЕР УКАЗАТЕЛЯ ИЛИ ССЫЛКИ

- * Теперь мы понимаем, что то, что мы зовем «указателем» или «ссылкой» – является **обычным числом-индексом**, где в памяти лежат наши данных
- * Размер этого числа **не может превышать формальный размер шины** и зачастую совпадает с ним
- * Например, указатель у программы для 64-битной системы – **это целое беззнаковое размера 64 бита**
- * Мы не можем сделать больше, но **можем сделать меньше...**

ЗАЧЕМ
ДЕЛАТЬ
МЕНЬШЕ?



НЕСКОЛЬКО ПРИЧИН

- * Это позволяет запускать старые программы (например, 32-битные)
- * Такие программы **не смогут использовать всю память системы**
- * Например, движок Fallout 3/Oblivion не может использовать **более 4 гигабайт памяти** и это зачастую проблема для мододелов
- * С другой стороны 32-битная программа и **требовать памяти будет меньше** (ведь меньше размер указателя, а их в программах много)
- * Поэтому такую оптимизацию используют на [практике](#)
- * Попробуйте в Chrome *performance*.memory.jsHeapSizeLimit

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОСТАРАЙТЕСЬ ПОНЯТЬ СУТЬ

- * Процессор и память – **два разных устройства**
- * Память для процессора это просто большой «**UInt8Array**»
- * Адрес значения в памяти – это **индекс первого байта, где лежат эти данные**
- * Указатель/ссылка – это значение-число, которое **содержит адрес другого значения в памяти**

А ЕЩЕ ПРОЦЕССОР ИМЕЕТ СВОЮ ПАМЯТЬ

- * Это **небольшие ячейки памяти**, обычно, размером 32 или 64 бита
- * Такие ячейки называются **регистрами** процессора
- * В отличии от DRAM, **регистры являются частью самого процессора**
- * Каждый процессор **имеет фиксированное количество регистров**
(согласно архитектуре и конкретной модели процессора)
- * **Часть регистров отведены под конкретные задачи**,
а другая часть – под любые данные

ЗАЧЕМ ПРОЦЕССОРУ НУЖНЫ РЕГИСТРЫ?



ЧТОБЫ РАБОТАТЬ С ДАННЫМИ

- * Процессор умеет выполнять инструкции **только для данных загруженных в регистры**
- * Фактически, загружаемая из DRAM информация переносится в регистры и **уже потом с ними можно что-то поделаться**
- * Регистр определяет **максимальный объем информации**, с которой процессор может работать «за раз»
- * За **выполнение инструкций или общение с памятью** отвечают свои специальные регистры

ПРОЦЕССОР БЫСТРЕЕ DRAM ПАМЯТИ

- * Несмотря на появление новых технологий памяти, **разрыв между процессором и памятью огромен** (в сотни раз)
- * Разница только **растет со временем**
- * А вот регистры **работают на одной скорости с ядром процессора**
- * Пока процессор **ждет данные из оперативной памяти** – он простаивает...

ЗНАЧИТ НУЖНО
ИСПОЛЬЗОВАТЬ
РЕГИСТРЫ
ВМЕСТО DRAM?



ДА, НО ТУТ ЕСТЬ НЮАНС

- * Регистров мало и **они могут хранить лишь очень малый объем информации** (от сотни байт до нескольких кибибайт)
- * А оперативная память может хранить сотни гигабайт!
- * Действительно, можно написать программу максимально использующую регистры, **но это не всегда возможно и крайне сложно**
- * Взаимодействие с регистрами возможно на языках ассемблера или косвенно в низкоуровневых ЯП, например C/C++ или Rust
- * Всем остальным **остается надеяться только на оптимизации**

ЧТО ДЕЛАТЬ
ТО В ИТОГЕ?



ПРИДУМАЛИ ЕЩЕ ОДНУ ПАМЯТЬ

- * Она является частью процессора, но **медленней регистров**
- * Зато она может содержать куда больше информации (от сотен кибибайт до гигабайта) и **все еще значительно быстрее DRAM**
- * Такую память называли **кэш-памятью** или «**кэшем**» процессора

**А КАК ЭТО
РАБОТАЕТ?**



ДАННЫЕ ИЗ DRAM ЗАГРУЖАЮТСЯ В КЭШ

- * Когда процессор хочет работать с данными из оперативной памяти, он **предварительно должен проверить, а нет ли этих данных в кэше**
- * Если их нет (кэш-промах), то **будет честный поход в оперативную память**
- * **Загруженные данные сохранятся в кэше**, а уже потом попадут на регистр
- * Когда за данными не надо ходить в память (кэш-попадание) – **наша программа будет работать значительно быстрее**
- * **Именно это мы и видели** в примере в начале лекции

НО ПОЧЕМУ
ПОРЯДОК
ОБХОДА ТАК
ВЛИЯЕТ?



ТУТ ЕСТЬ НЕСКОЛЬКО ПРИЧИН

- * Размер кэша **значительно меньше** возможного размера DRAM
- * Значит, придется **постоянно загружать и выгружать данные** из кэша
- * Этой логикой занимается контроллер кэша
- * Но помимо самих данных из DRAM, в кэше **нужно хранить их адреса**
- * Чтобы не хранить адрес каждого байта, **их объединяют в группы** – кэш-линии
- * Тогда нужно будет хранить **только адрес кэш-линии**
- * Размер линии на x86 равен 64 байта

А ТЕПЕРЬ ГЛАВНЫЙ ВЫВОД

- * Если данные лежат рядом в памяти, то **они с высокой долей вероятности будут лежать рядом и в кэше**
- * Когда мы обходим массив последовательно, то у нас получается **минимизировать обращение к DRAM** за счет использования кэша
- * Когда мы «прыгаем по массиву», то ситуаций, **когда данных в кэше нет становится куда больше**
- * Так как **кэш не может вместить всю память** и должен периодически выгружать неиспользуемое

1 Загружаем кэш-линию с [0,0-7]

✗ Используем ТОЛЬКО [0,0]

2 Загружаем кэш-линию с [1,0-7]

✗ Используем ТОЛЬКО [1,0]

3 Старая линия вытесняется из кэша

4 Когда вернемся к [0,1] – её уже НЕТ в кэше!

Результат:

● Каждая загрузка кэш-линии дает 1 полезный элемент

● 7 из 8 элементов загружены ВПУСТУЮ

● Линии вытесняются ДО того, как мы к ним вернемся

✉ Эффективность: 12.5% использования загруженных данных

↓ ОБХОД С ШАГОМ (по столбцам)

Порядок обхода: [0,0] → [1,0] → [2,0] → [3,0] → ...

1 Загружаем кэш-линию с [0,0-7]

✓ Используем все 8 элементов

2 Кэш-линия остается в кэше

✓ Следующая линия не вытесняет предыдущую

Результат:

● Кэш-линии используются полностью

● Вытеснение происходит только после использования

✉ Эффективность: 100% использования загруженных данных

→ ПОСЛЕДОВАТЕЛЬНЫЙ ОБХОД (по строкам)

Порядок обхода: [0,0] → [0,1] → [0,2] → ...

РЕФЛЕКСИРУМ

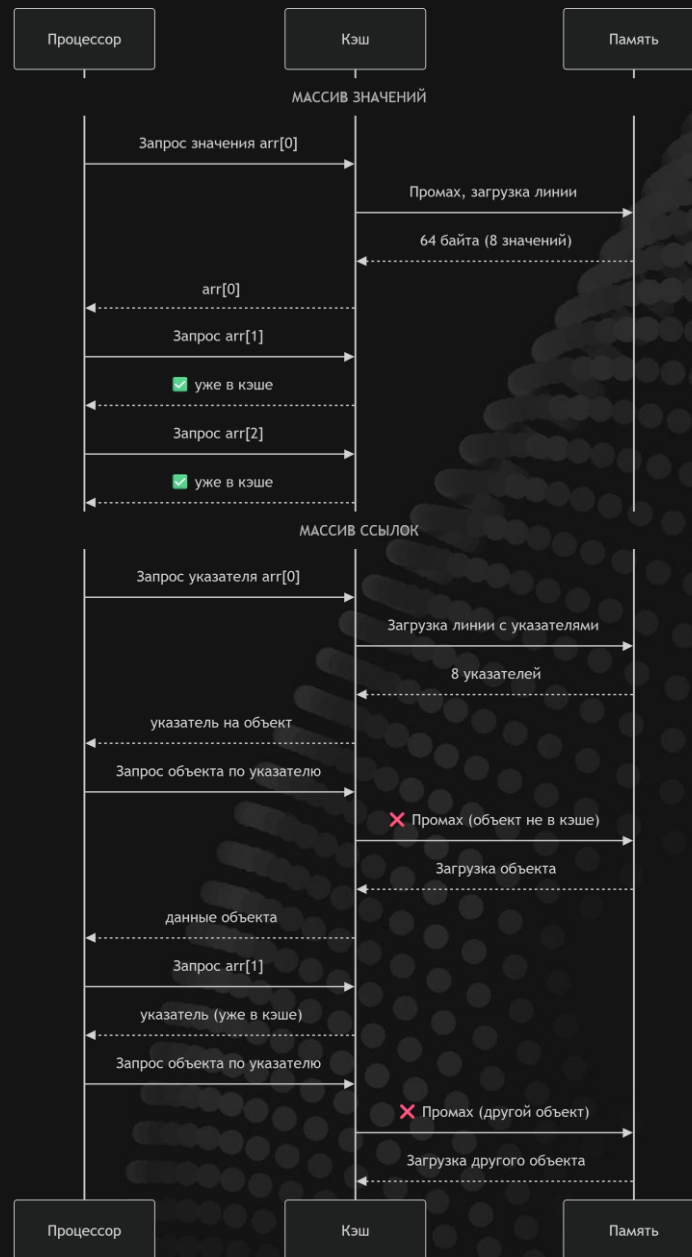
- * С последовательным подходом мы **полностью используем кэш-линию после её загрузки**
- * А в другом случае мы **используем только один элемент после загрузки линии** и переходим к другой линии
- * Когда мы возвращаемся к следующим данным из уже загруженной линии, **то она может быть уже выгружена из кэша**
- * Поэтому, **чем больше массив, тем сильнее негативный эффект**

ПОЧЕМУ С ОБЪЕКТАМИ РАЗНИЦА БОЛЬШЕ?



В JS ЛЮБОЕ ЗНАЧЕНИЕ – ЭТО УКАЗАТЕЛЬ

- * Но некоторые указатели таковыми на поверку не являются и **содержат сами данные**
- * К тому же, JIT может и для ссылочных типов **оптимизировать их в более эффективное представление** по значению
- * А вот с объектами все сложнее, так как они с одной стороны **реально хранятся по ссылке** на сегмент памяти «куча»
- * А с другой, **изменяемы и тесно связаны с механизмом сборки мусора**
- * Поэтому оптимизировать их как строки или числа сложнее



РЕЗЮМИРУЕМ

- * Ссылочная модель JavaScript **усложняет использование кэша-процессора**
- * Нам **приходится полагаться на JIT**, что он сможет эффективнее переконфигуровать наши данные
- * Или мы **можем использовать типизированные массивы**

КАК ТИПИЗИРОВАННЫЕ МАССИВЫ ПОМОГУТ?



ТРИ ПРИЧНЫ

- * Мы можем контролировать размер одного элемента массива и **класть больше значений в одну кэш-линию**
- * Содержимое типизированных массивов **всегда хранится по значению** [ссылку сделать можно, но только явно]
- * А значит **повышается локальность кэша**
- * **Удаление или изменение содержимого** типизированного массива не тревожит сборщик мусора

ЭТО **ТАК**
ВАЖНО?

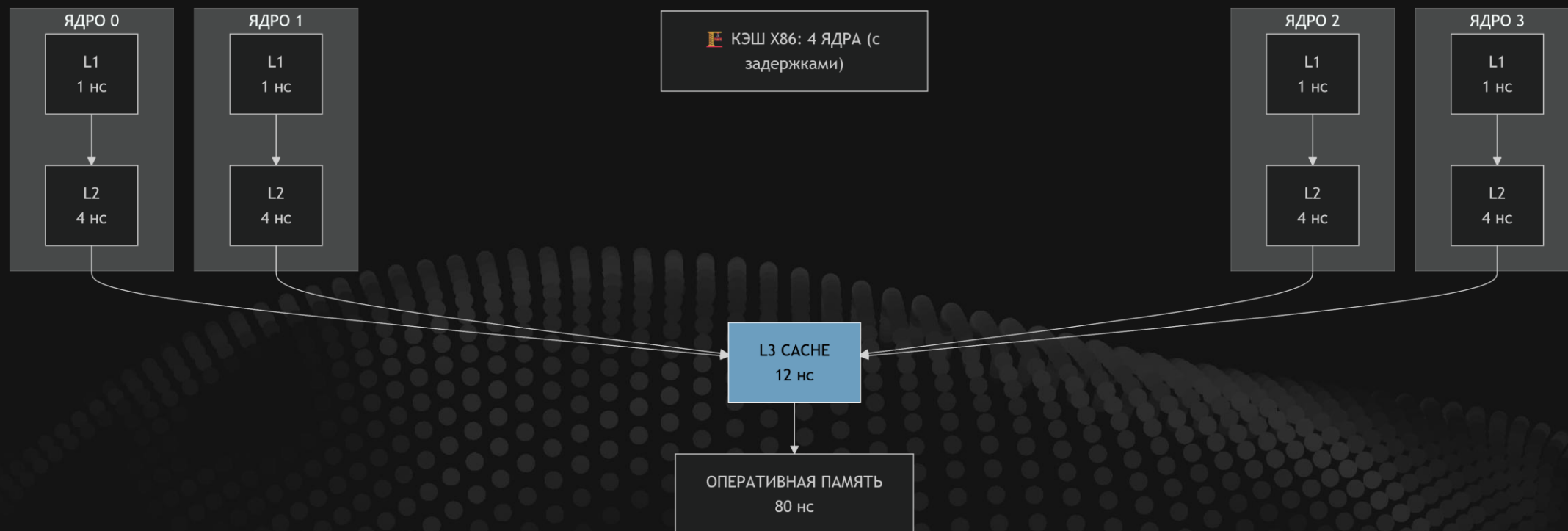


ЕСЛИ У ВАС «ТЯЖЕЛОЕ» ПРИЛОЖЕНИЕ – ДА

- * Например, нагруженный веб-сервер, игра или графический редактор
- * Грамотное использование типизированных массивов позволит добиться **многократного прироста производительности**
- * JIT вам скажет спасибо

ЧТО ЕЩЕ ВАЖНО СКАЗАТЬ ПРО КЭШ

- * **Современные процессоры** используют многоуровневые кэши
- * Отличаются уровни **максимальным объемом памяти и скоростью работы**
- * Некоторые уровни **общие для всех ядер процессора**,
а некоторые – у каждого свои
- * Это важно при написании **многопоточных программ с общей памятью**
- * В современных x86 процессорах используется 3 уровня кэша



КАЖДОЕ ЯДРО МОЖЕТ РАБОТАТЬ ПАРАЛЛЕЛЬНО

- * Но они все равно работают с общей памятью
- * Отдельные кэш-уровни на ядро позволяют повысить **изолированность и параллельность выполнения**
- * Общий кэш нужен, чтобы не ходить в память лишний раз **(если уже другое ядро загрузило эти данные)**
- * В реальности все куда сложнее, поэтому один из лучших паттернов – **это не использовать общие ресурсы в многопоточном коде**

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

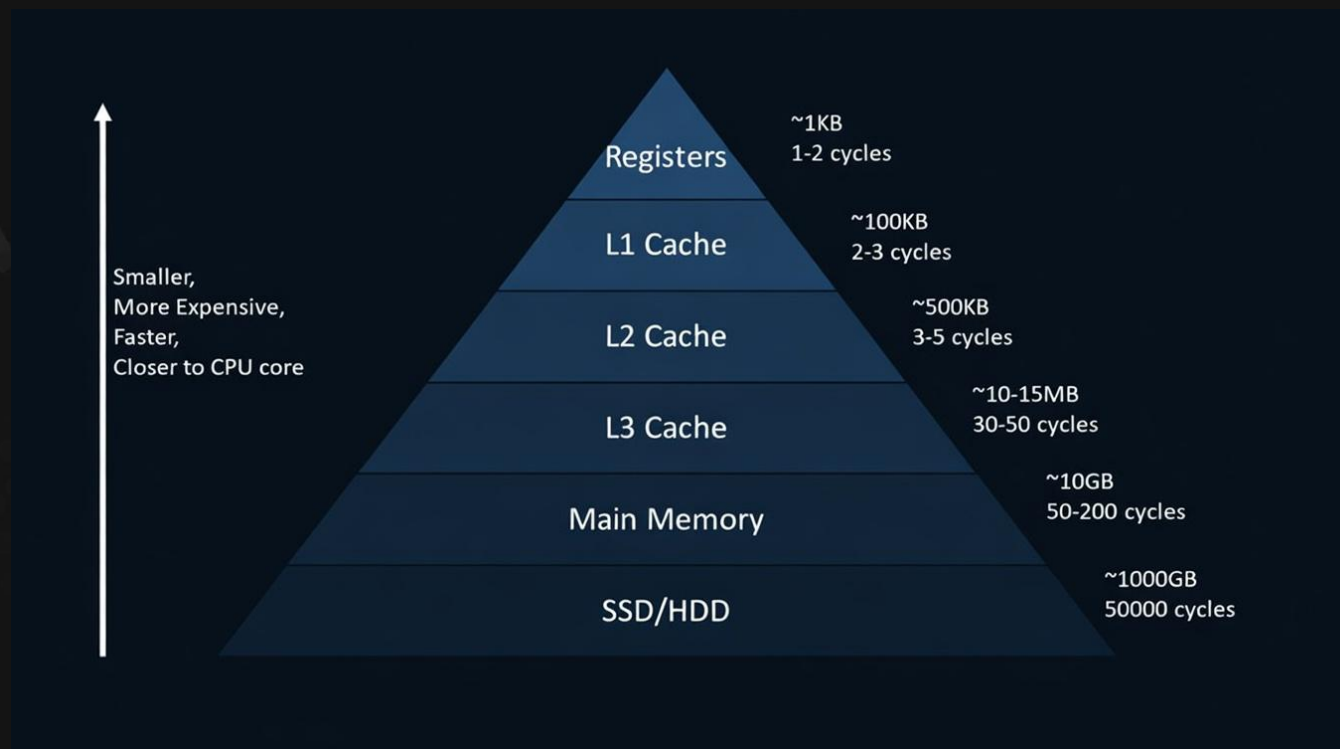
\$160.00

СТОЯНА, СТОЯНААА!

ПОЙМИТЕ СУТЬ

- * Если данные лежат рядом в памяти, то и в кэше они тоже будут лежать рядом
- * Если собираетесь хранить данные в массиве,
то храните их в том порядке, в каком будете чаще всего обходить
- * Массив значений лучше для кэша, чем массив ссылок
- * Типизированные массивы позволяют тонко контролировать работу кэша

ДЕРЖИТЕ В ГОЛОВЕ



ДЕЛО НЕ ТОЛЬКО В КЭШЕ

- * Эффективная работа кэша это **50-60% оптимизации вычислений процессором**
- * Но есть еще вещи, про которые нужно иметь ввиду

ЯДРО ПРОЦЕССОРА ПОСЛЕДОВАТЕЛЬНО ВЫПОЛНЯЕТ ОПЕРАЦИИ

- * Но когда встречается инструкция «с условием», то процессор «на распутье»
- * Если условие положительно, то надо **выполнить все «вложенные инструкции»**, а если отрицательное – нет
- * Вычисление условия может потребовать новый поход в память, а **тем временем процессор вынужден ждать**
- * Либо процессор **может «прогреть» память под положительный сценарий**, пока ждет загрузки условия
- * Если условие **окажется положительный** – получаем прирост

НАПРИМЕР

```
// 1. Без условий (baseline)
function rowMajorNoCondition(array, size) {
  let sum = 0;

  for (let i = 0; i < size; i++) {
    for (let j = 0; j < size; j++) {
      const index = i * size + j;
      sum += array[index];
    }
  }

  return sum;
}
```

```
// 2. Предсказуемое условие
// (четность - легко предсказать)
function rowMajorParityCondition(array, size) {
  let sum = 0;

  for (let i = 0; i < size; i++) {
    for (let j = 0; j < size; j++) {
      const index = i * size + j;
      const value = array[index];

      if (index % 2 === 0) { // Четные индексы
        sum += value;
      }
    }
  }

  return sum;
}
```

```
// 3. Нестабильное условие
// (случайное - трудно предсказать)
function rowMajorRandomCondition(array, size) {
  let sum = 0;

  for (let i = 0; i < size; i++) {
    for (let j = 0; j < size; j++) {
      const index = i * size + j;
      const value = array[index];

      if (value > 50) { // Случайное условие
        sum += value;
      }
    }
  }

  return sum;
}
```



1000x1000 (1 000 000 элементов)

Без условий: 0.60ms

Четность: 0.75ms (1.24x медленнее)

Случайное: 3.07ms (5.11x медленнее)

Разница: случайное медленнее четности в 4.12x

2000x2000 (4 000 000 элементов)

Без условий: 2.38ms

Четность: 2.95ms (1.24x медленнее)

Случайное: 12.38ms (5.21x медленнее)

Разница: случайное медленнее четности в 4.20x

3000x3000 (9 000 000 элементов)

Без условий: 4.99ms

Четность: 6.68ms (1.34x медленнее)

Случайное: 28.03ms (5.61x медленнее)

Разница: случайное медленнее четности в 4.20x

ВЫВОДЫ

- * Условия внутри цикла могут драматически повлиять на производительность нашего кода
- * Процессоры пытаются «предугадать» ход вычисления и «прогреть» инструкции – когда это получается, то мы получаем заметный прирост производительности
- * Алгоритмы работы «оракула» связаны с конкретным процессором и его микрокодом

ЧТО ВАЖНО ДОБАВИТЬ

- * Предсказать (или «оракул») использует самообучающуюся модель – то есть он **не бездумно делает предположение о положительном условии**
- * Сами применяемые оптимизации могут включать в себя как прогрев памяти (**prefetching**), так и выполнение кода (**speculative execution**)
- * У таких оптимизаций есть цена в безопасности
- * Именно спекулятивное исполнение привело к появлению знаменитых уязвимостей **Meltdown (2018)** и **Spectre (2018)**

КАК ЭТО РАБОТАЛО

- * Атакующий **заставлял процессор спекулятивно прочитать секретные данные** (например, пароль из ядра системы)
- * Процессор читал пароль, выполнял с ним вычисления (например, умножал на что-то) — это влияло на состояние кэша
- * Хотя процессор потом понимал, что «не имел права» это делать, и откатывал вычисления, **следы в кэше** оставались
- * Атакующий измерял время доступа к разным участкам памяти и по этим следам **восстанавливал пароль**

ЭТО ПОФИКСИЛИ, НО ОПАСНОСТЬ ОСТАЛАСЬ

- * Intel и AMD выпустили обновления микрокода процессоров с исправлением уязвимостей
- * В некоторых случаях это привело к просадке производительности
- * Нет гарантии, что похожие проблемы не всплывут в будущем

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОЙМИТЕ СУТЬ

- * Каждый **if** в вашем коде – это потенциальный тормоз
- * Помните про этот момент и старайтесь писать код «проще»

ЕСТЬ ЕЩЕ ОПТИМИЗАЦИИ

- * Если кэш и предсказание выполнялись на уровне процессора, то есть ряд оптимизаций, **которые делает компилятор или среда исполнения**
- * Оптимизаций много, но сводятся они к тому, что **ваш код преобразуется в другой, более эффективной**
- * В JS такими оптимизациями занимается JIT
- * Важнейшей из таких оптимизаций является **векторизация**

SIMD ИНСТРУКЦИИ

- * Большинство инструкций процессора работают по принципу:
одна инструкция – одни данные (например, сложи 2 числа)
- * Но есть такие инструкции, которые работают как:
одна инструкция – много данных (SIMD)
- * Например, **сложи массив из 8 чисел с другим таким массивом**,
где результатом будет новый массив
- * Особенность таких инструкция в том, что они работают атомарно:
одна инструкция, а не цикл из инструкций

ВЕКТОРИЗАЦИЯ

- ✱ Смысл оптимизации в том, чтобы переписать цикл таким образом, чтобы **уменьшить количество итераций за счет применения SIMD инструкций**
- ✱ Если такая оптимизация возможна, то можно получить **значительный прирост производительности**

```
// Скалярная версия (без векторизации)
function addArraysScalar(a, b, result) {
  for (let i = 0; i < a.length; i++) {
    result[i] = a[i] + b[i]; // ОДНА операция за такт
  }
}
```

```
// Векторная версия (с SIMD.js)
function addArraysSIMD(a, b, result) {
  // SIMD обрабатывает 4 числа за одну операцию
  for (let i = 0; i < a.length; i += 4) {
    // Загружаем 4 числа из массива A в вектор (128 бит)
    const vecA = SIMD.Float32x4.load(a, i);

    // Загружаем 4 числа из массива B в вектор
    const vecB = SIMD.Float32x4.load(b, i);

    // Выполняем 4 сложения за 1 команду
    const vecResult = SIMD.Float32x4.add(vecA, vecB);

    // Сохраняем результат обратно в массив
    SIMD.Float32x4.store(result, i, vecResult);
  }
}
```


ЗАМЕЧАНИЕ К ПРИМЕРУ

- * Стандарт [SIMD.js](#) все еще является драфтом и использовать такие инструкции явно не выйдет
- * Остается только полагаться, что JIT сможет применить эту оптимизацию
- * Но мы можем ему помочь применить эту оптимизацию:
использовать типизированные массивы и не делать условия внутри цикла
- * А вот WebAssembly уже [ограниченно поддерживает](#) SIMD явно
- * Ограниченность связана с переносимостью
(не все процессоры поддерживают все виды SIMD инструкций)

second monitor

\$0.00 (0%)

50.00

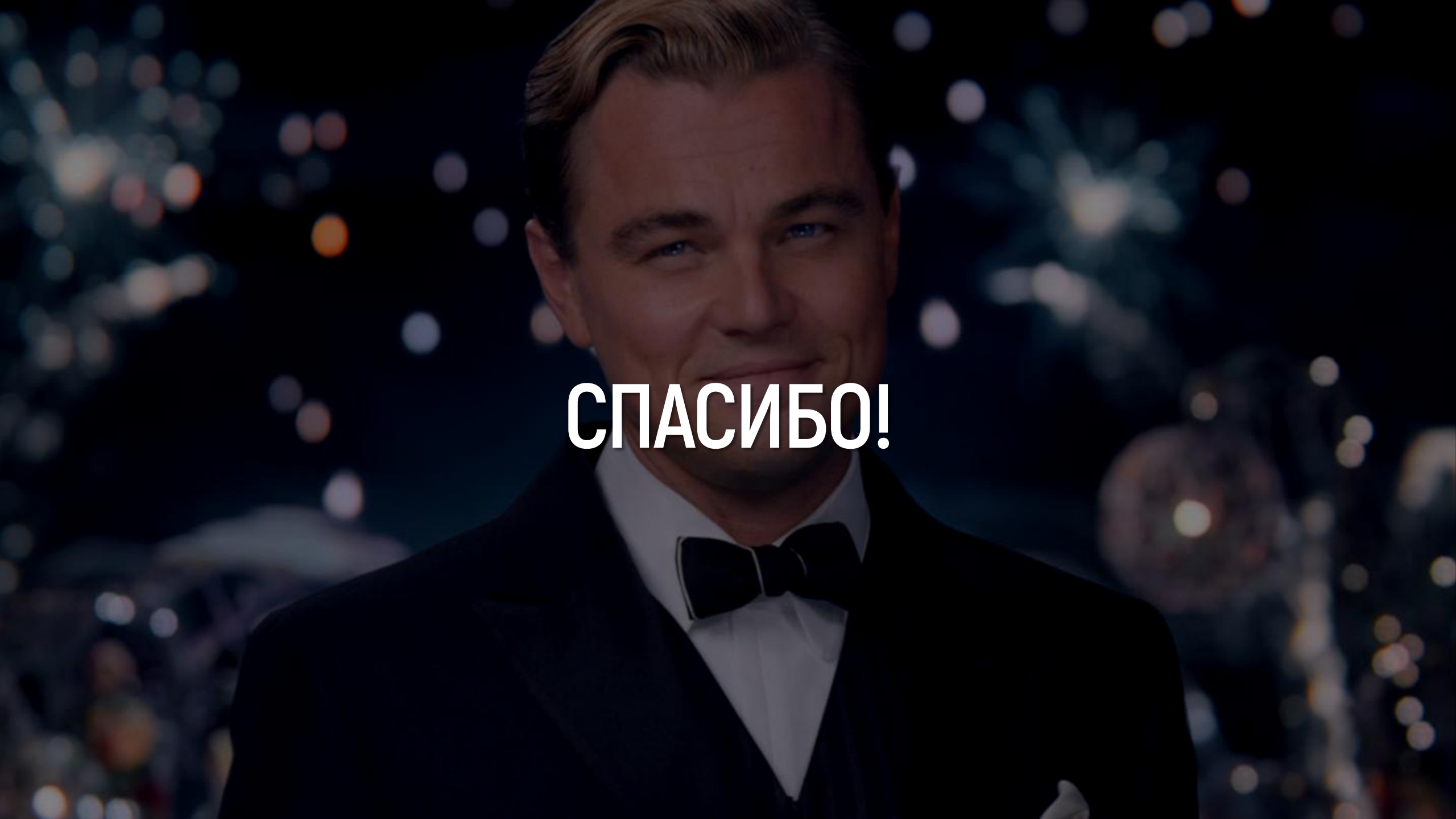
224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОДВЕДЕМ ИТОГИ

- * Процессор и память являются **важнейшей частью любого компьютера**
- * В силу физической архитектуры, **процессоры работают намного быстрее памяти**
- * Чтобы **избежать «простоев» процессора из-за обращений к памяти** придумали кэш-память процессора
- * Понимание принципов работы кэша крайне важно для написания **эффективного кода даже на таком высокоуровневом ЯП как JS**
- * Помимо использования кэша, процессоры и компиляторы **применяют другие оптимизации**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!